

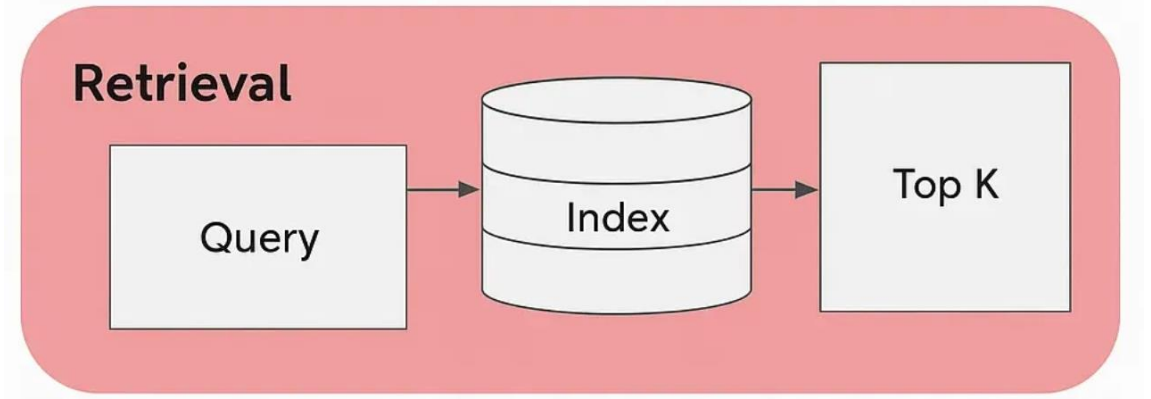


University
of Exeter

COMM117

Large Language Models and Application

Morden Retriever setting



- **Top-k**: start **5** (raise to **10** if misses; lower if noisy/costly)
- **Threshold (similarity)**: start **0.78** (raise 0.82 = cleaner; lower 0.74 = fewer empty retrieves)
- **Filters**: where to focus? (**region/version/department**) to avoid wrong doc → department="Finance" AND region="UK" AND version_date >= 2025-01-01
- **Hybrid**: **0.7 vector / 0.3 keyword** (shift toward keyword for IDs/codes)

Three levels of RAG evaluation

- Retrieval level: Retrieval quality
- Generation level: Answer correctness Faithfulness
- System level: Latency, Cost

Evaluation metrics recap

Retrieval level

Hit Rate → was any **relevant evidence** found?

Recall@k → how much relevant evidence was found? **(complete)**

Precision@k → was retrieval **clean or noisy**?

F1@k → balance of **coverage and precision**

MRR → was the **first relevant** chunk ranked early?

NDCG → was the **overall ranking good**?

Generation-level evaluation

1. Exact Match (EM)/ F1 score

Checks whether the generated answer exactly matches the gold answer./Measures token overlap between generated answer and reference answer.

strict. → Good for short factual QA.

2. ROUGE/ BLEU

Summarisation/ translation

3 . BERTScore / semantic similarity

Compares semantic similarity, not just exact wording.

4. Human Evaluation/ QA/ LLM as judge

Faithfulness check

System – level evaluation

Did the model use the evidence correctly?

- Latency : Average response time
- Cost: Average token cost, Cost per query
- **Robustness**: answer accuracy on noisy queries
- **Reliability**: the same question asked several times

Synthetic test generation

Generating synthetic RAG test cases from the knowledge base

- start from a chunk in the knowledge base
- ask an LLM to generate:
 - a question
 - a good answer
- this creates grounded in-domain evaluation examples

but still needs human review for quality

Introduction Today

- RAG Variants
- Chaining, tool use, and agent frameworks- failure modes

From Paper 2020 to Modern RAG Systems

- **Reranking** improve precision before generation.
- Modern stacks add **hybrid search** (dense + sparse).
- **Observability and evaluation** are now first-class concerns.

Advanced RAG

Basic RAG is useful, but real systems usually need **more than a single retrieve-then-generate step.**

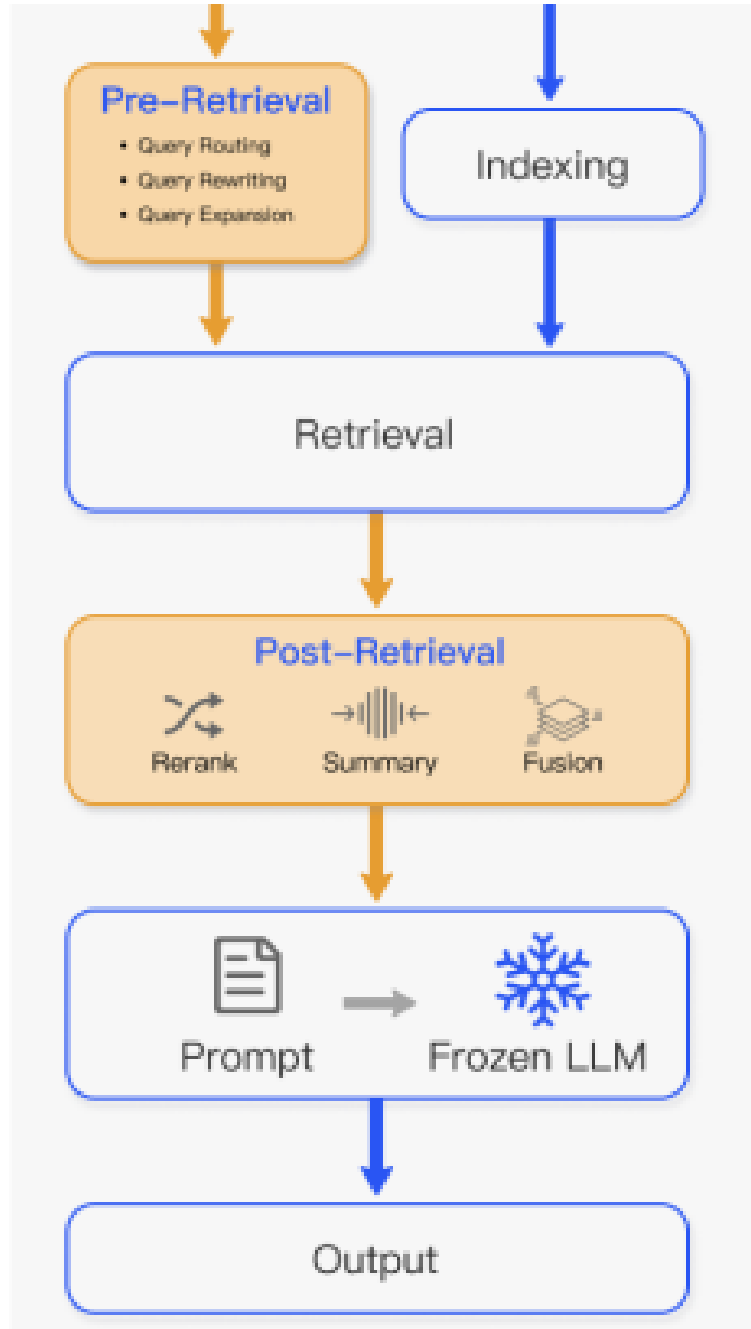
- **Advanced RAG** improves the baseline pipeline with stronger retrieval and generation components
- Typical additions:
 - hybrid retrieval
 - reranking
 - better query processing
 - richer context construction
 - feedback / correction loops



Goal:

- higher accuracy
- better robustness
- better performance in real applications

Advanc



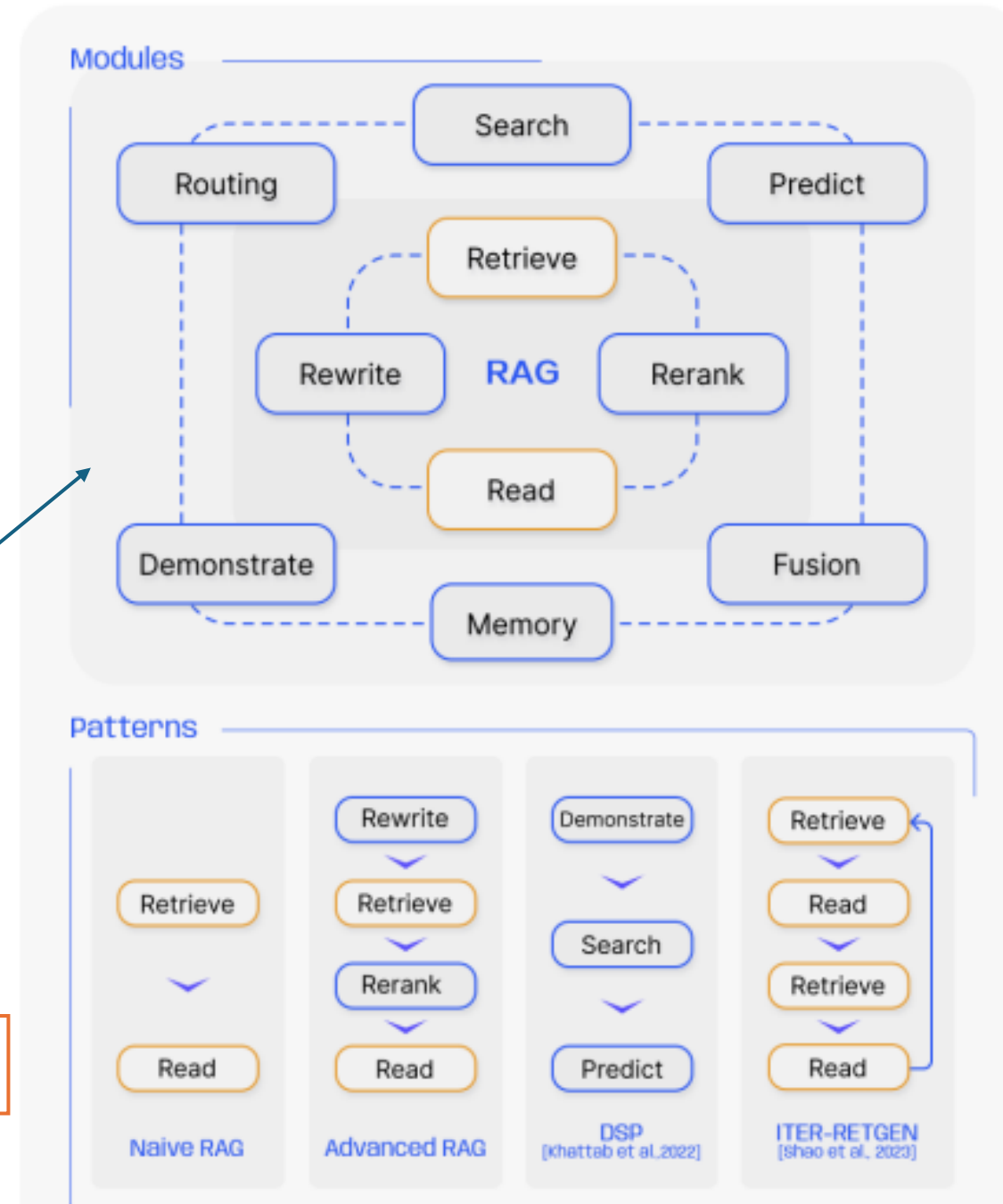
we are still doing RAG, but we improve one or more steps in the pipeline.

Modular RAG

As RAG becomes more advanced, it is useful to think of it as **a set of modules** rather than one fixed pipeline.

a library of possible modules

some **example patterns** built from them



RAG Variants

- Naive RAG 
- Hybrid RAG 
- Reranked RAG 
- Query-transformed RAG
- Parent-child / hierarchical RAG
- Multi-hop RAG
- Corrective / self-reflective RAG
- Agentic / tool-augmented RAG

RAG Variants - Query transformation

Before retrieval:

- rewrite the query
- expand the query
- decompose the query
- add metadata constraints

Why:

- user queries are vague
- documents are written in different vocabulary
- help standardize and enhance user inputs, making them more digestible for our retrieval system.

Examples:

- “**money back** for downloaded file” → “**refund** for digital product”
- “0xA93 issue” → “authentication failure error code 0xA93”

RAG Variants - Metadata-filtered RAG

Does **not** mainly rewrite the query.

Instead, it restricts **where retrieval is allowed to search**.

Examples:

- only search documents from **UK HR**
- only search **2025 policy docs**
- only search **product A**
- only search **finance reports**

- Use an LLM call to extract entities.
- Use zero-shot entity detection models like [GLiNER](#). For example, given below are the Gliner extracted entities for our example query.

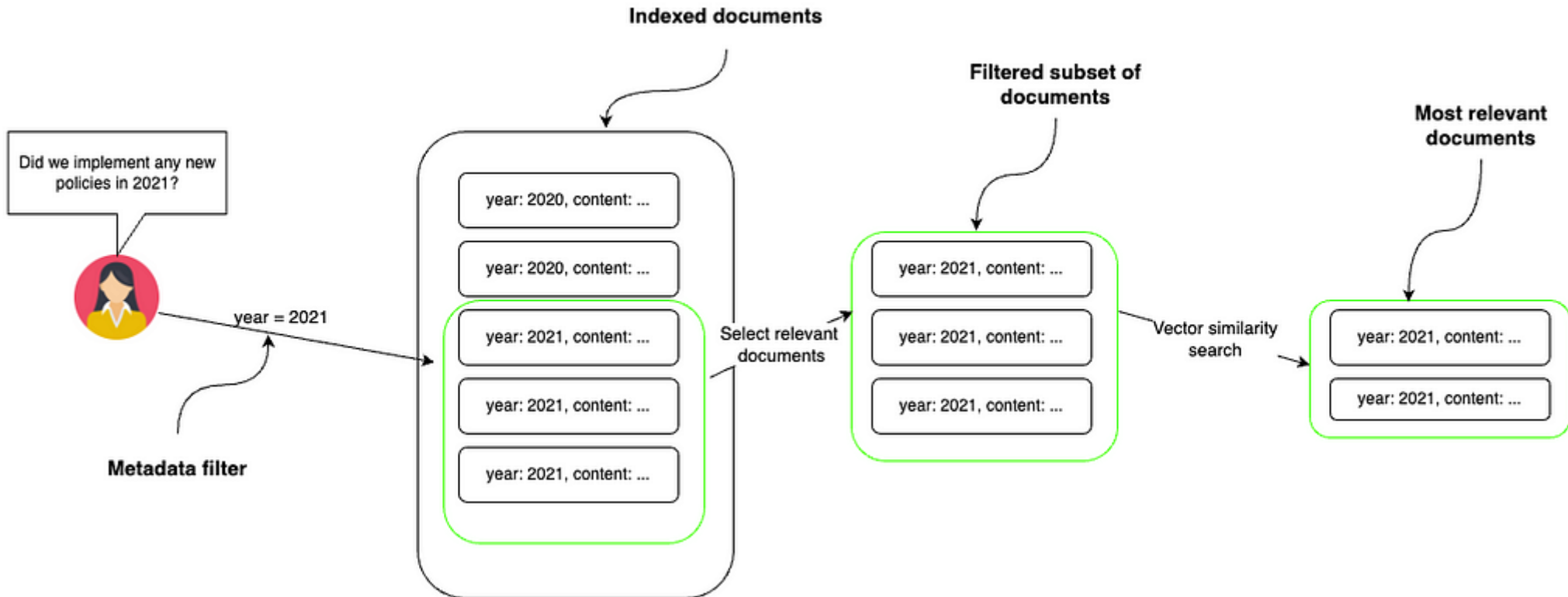
RAG Variants - Metadata-filtered RAG

Does not mainly rewrite the query.

Instead, it restricts **where retrieval is allowed to search**.

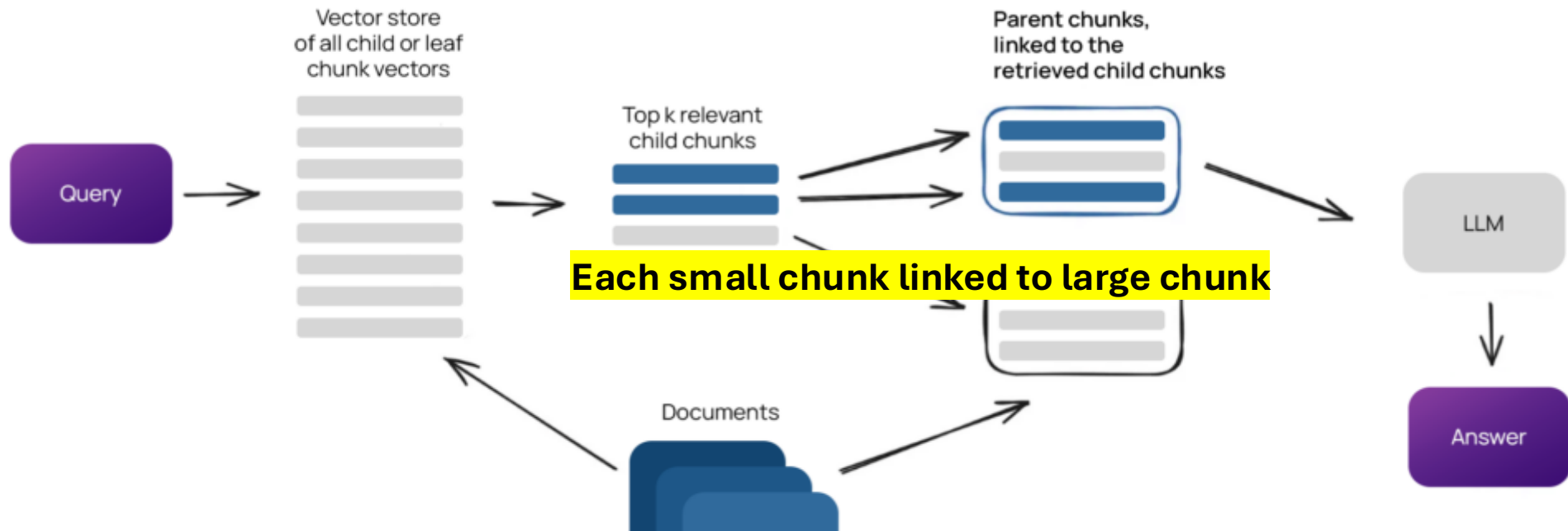
Example

- 01
- 01
- 01
- 01



RAG Variants - Parent-child / hierarchical RAG

- retrieve small chunks for precision —→ improve matching
- return larger parent section for answer generation —→ gives enough information to answer



RAG Variants - Parent-child / hierarchical RAG

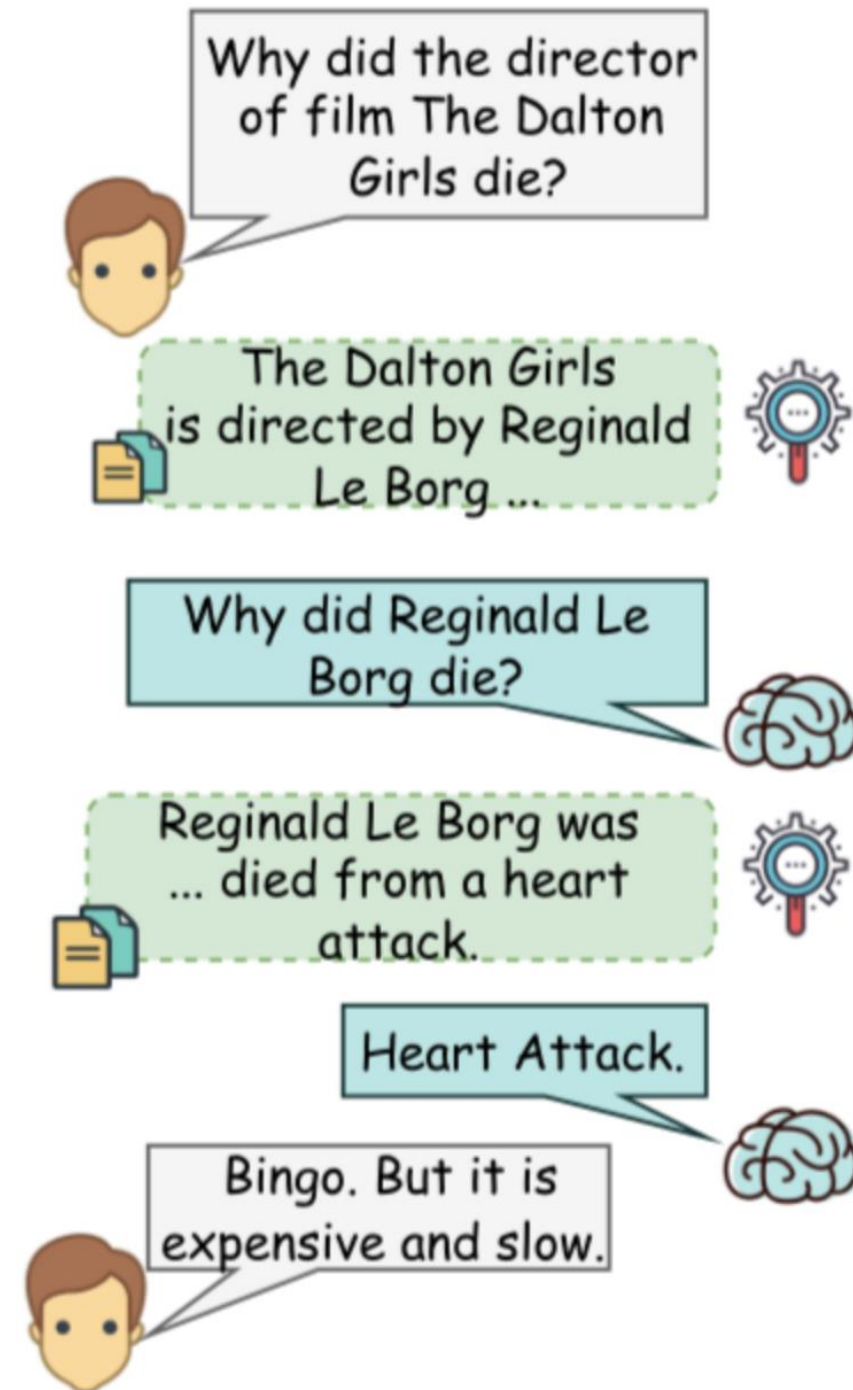
- Create smaller chunks for embedding, which are **optimized** for the **limited context length of most embedding models**.
- Each small chunk is linked to a larger chunk that contains more context.
- When you search, you're looking through the **smaller chunks**, which allows for more accurate retrieval.
- But when you **feed the results to your LLM** for reasoning, you provide the corresponding larger chunks.

- small chunks: precise but fragmented
- large chunks: complete but noisy
- hierarchical RAG tries to balance both

RAG Variants - Multi-hop RAG

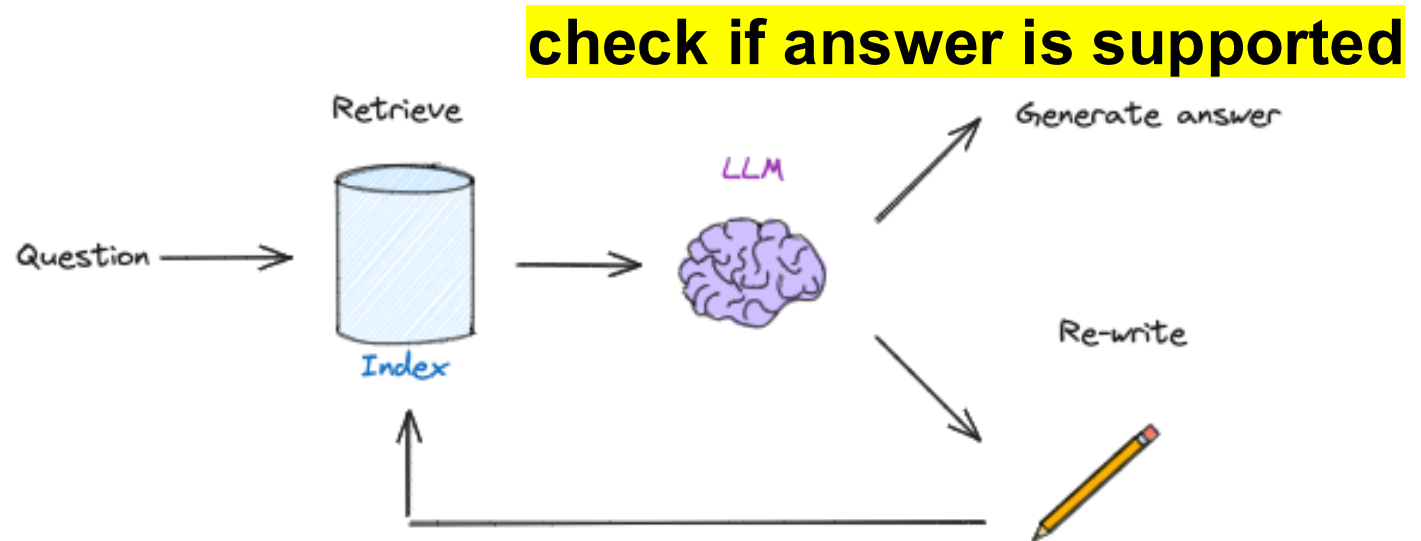
Some questions require **multiple pieces of evidence**:

- one chunk is not enough
- answer requires combining information across documents
- retrieve a fact, then retrieve a related fact



RAG Variants - Corrective / self-reflective RAG

- Reduce unsupported claims
- Improve faithfulness
- Make the system say “not enough information” when needed



Recap

- Naive RAG: query → retrieve → answer
- Hybrid RAG: hybrid retrieval → answer
- Reranked RAG: top 50 candidates → re-rank → answer
- Query-transformed RAG: rewrite query → retrieve → answer
- Filtered RAG : filter → retrieve → answer
- Hierarchical RAG: retrieve child → fetch parent → answer
- Multi-hop RAG: retrieve A → retrieve B → combine
- self-reflective RAG: retrieve → answer → verify

each variant adds explicit intermediate steps

Chaining

- Naive RAG: query → retrieve → answer
 - Hybrid RAG: hybrid retrieval → answer
 - Reranked RAG: top 50 candidates → re-rank → answer
 - Query-transformed RAG: rewrite query → retrieve → answer
 - Filtered RAG : filter → retrieve → answer
 - Hierarchical RAG: retrieve child → fetch parent → answer
 - Multi-hop RAG: retrieve A → retrieve B → combine
 - self-reflective RAG: retrieve → answer → verify
- 

Chaining

Many advanced RAG variants are no longer single-step systems.

They often require:

- intermediate decisions
- multiple retrieval stages
- evidence transformation
- verification or revision
- optional tool use

This turns RAG into a **chain of steps rather than a single retrieve-then-generate pipeline.**

Chaining

Chaining means structuring the system as **a sequence of explicit steps**, where the output of one step becomes the input to the next.

Chaining = multi-step workflows

Examples:

rewrite → retrieve → answer

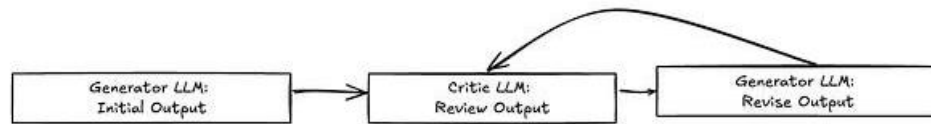
retrieve → rerank → answer

retrieve → retrieve again → combine

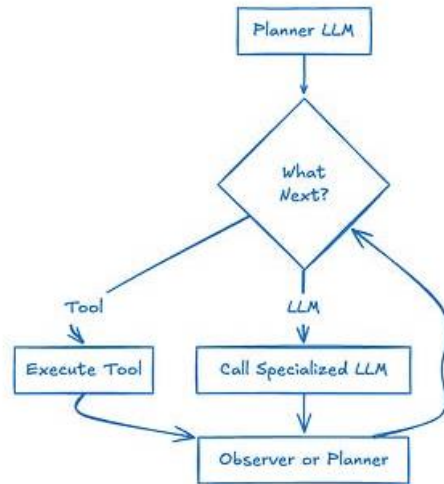
retrieve → answer → verify

Chaining

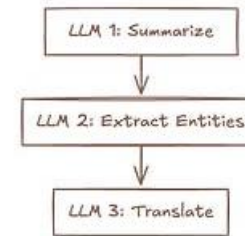
LLM Chaining: Self Refinement Loop



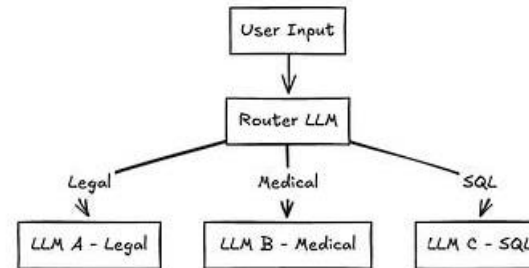
LLM Chaining: Agentic Loop



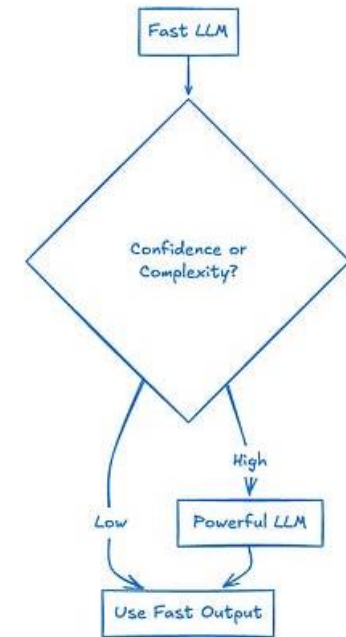
LLM Chaining: Sequential Pipeline



LLM Chaining: Router Dispatcher



LLM Chaining: Cascade Filter Escalate



Why Chaining

- breaks complex tasks into smaller steps
- improves control
- improves debugging
- lets us inspect intermediate outputs
- often better than one giant prompt

Chaining is not enough

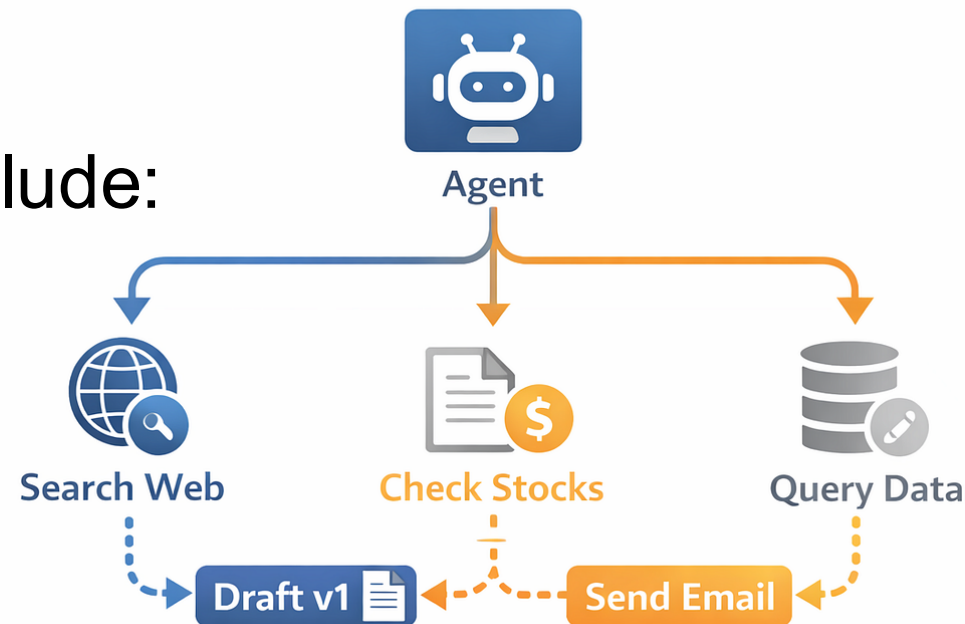
- some questions require structured data
- some require APIs or calculators
- some require choosing between multiple sources
- some require dynamic decisions, not one fixed chain

Tool use

How AI Agents Interact With the Real World

Retrieval is one tool other tools may include:

- SQL/database lookup
- calculator
- search engine
- API call
- code execution



Tool use helps when text retrieval alone is insufficient

“What was Q2 growth versus forecast?”

- retrieve forecast document
- query actual sales table
- compare

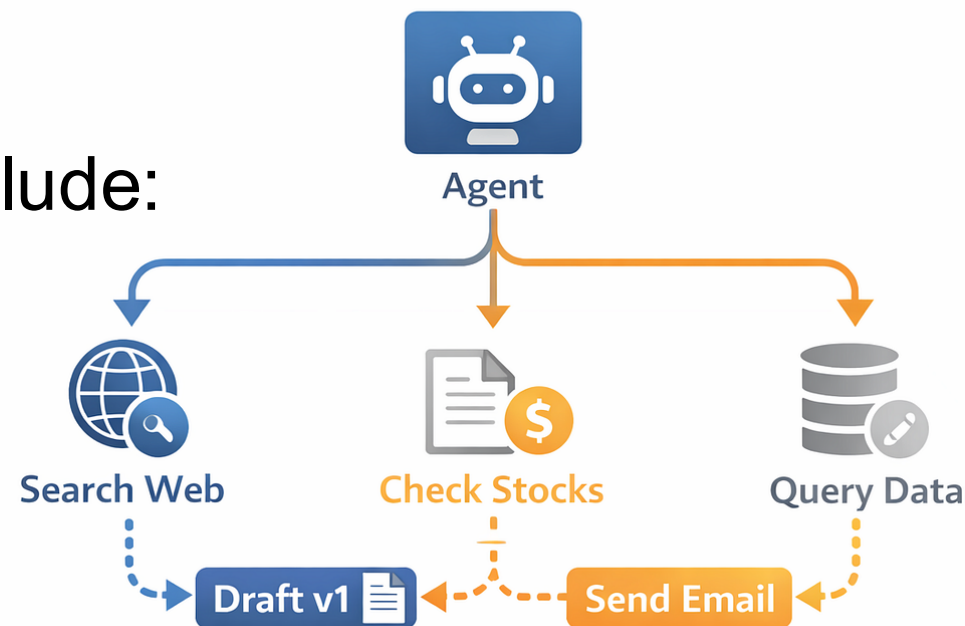
Tool use

How AI Agents Interact With the Real World

Retrieval is one tool other tools may include:

- SQL/database lookup
- calculator
- search engine
- API call
- code execution

Tool use helps when text retrieval alone is insufficient



“What was Q2 growth versus forecast?”

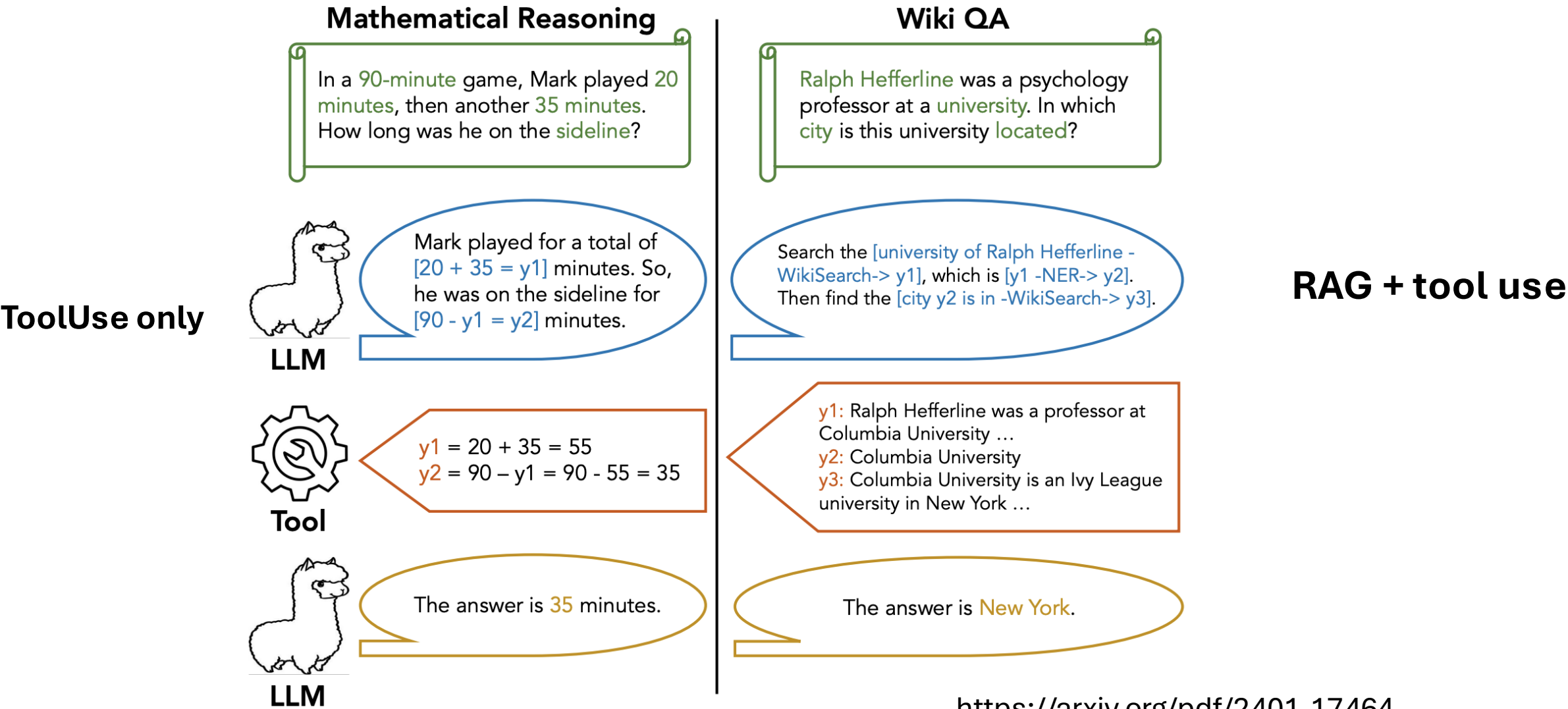
- retrieve forecast document
- query actual sales table
- compare

RAG VS Tool use

- use **RAG** for unstructured knowledge in documents
 - use **tools** for structured, up-to-date, or computed information
- many real systems combine both

- company handbook → RAG
- current stock price → API/tool
- revenue delta calculation → calculator/SQL

RAG VS Tool use



Failure modes in tool use

- wrong tool chosen
- wrong API parameters
- stale or missing data
- tool output misunderstood
- correct retrieval but incorrect computation

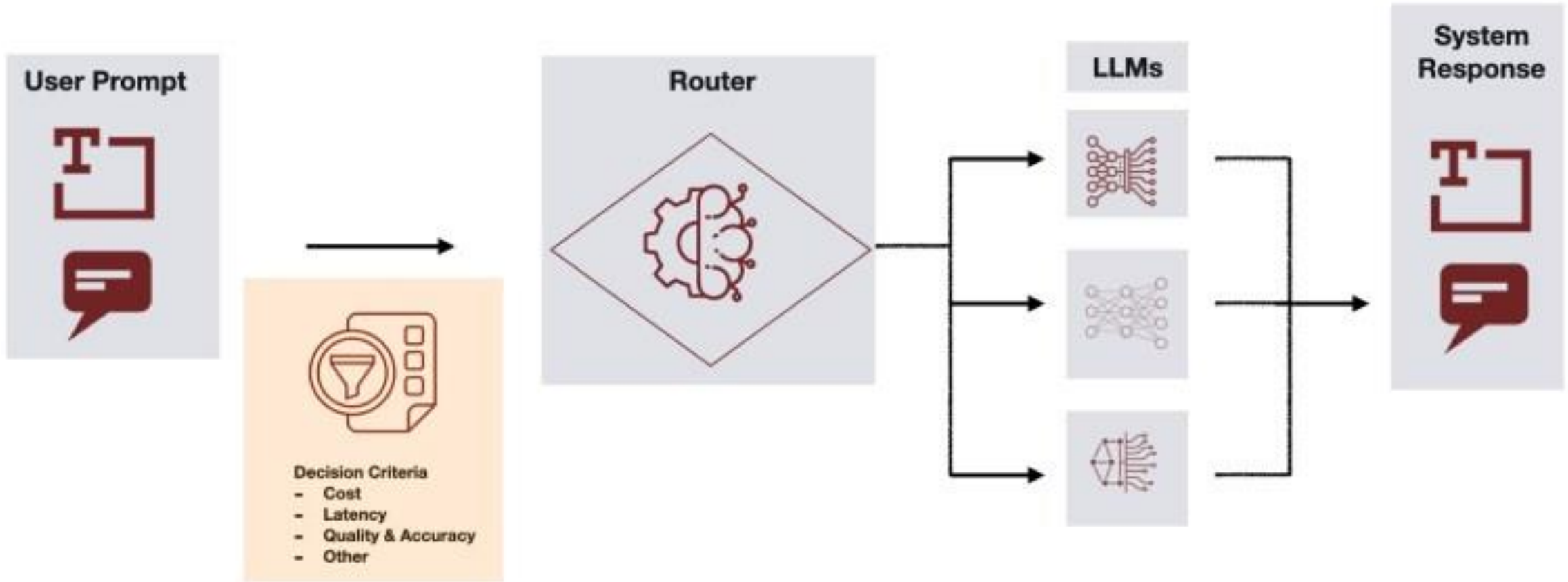
Router

A router decides **which tool or source should be used.**
it can choose:

- which retriever
- which knowledge base
- which tool
- which specialist agent

- ✓ HR question → HR docs
- ✓ finance question → finance DB
- ✓ technical issue → support knowledge base

Router



Router

Why routing matters in advanced RAG

- avoids searching the wrong corpus
- improves retrieval precision
- supports multi-source systems
- reduces noise and latency

Question:

“What was Q2 sales growth compared with forecast?”

Possible system:

RAG retriever gets the forecast from a report

router decides to use both the report retriever and the SQL tool

SQL tool gets actual Q2 sales

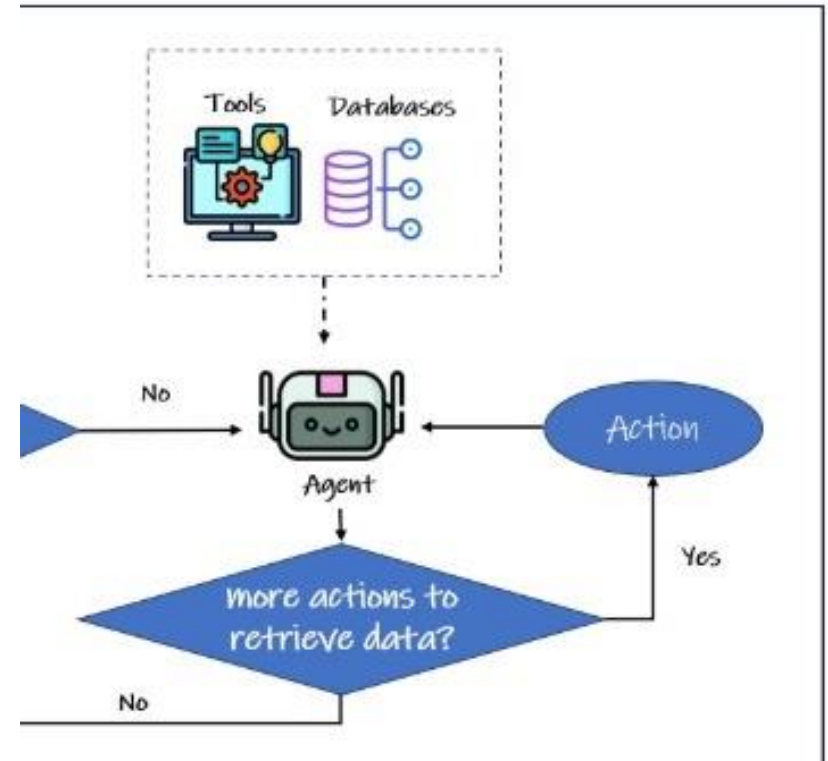
calculator computes the difference



Tooluse

From chaining to agents

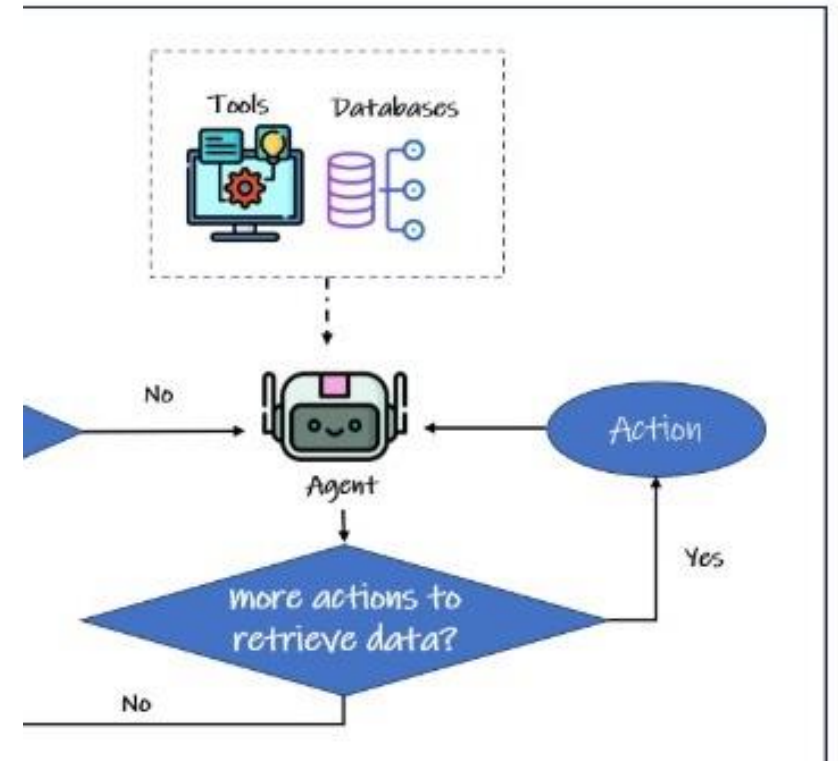
- **chain**: the steps are predefined
- **agent**: the system decides the next step dynamically
- agents are more flexible
- but also harder to control



Agent

the agent can decide to:

- **retrieve again**
- **rewrite the query**
- **route to a different source**
- **call a tool or API**
- **use structured data**
- **verify before answering**

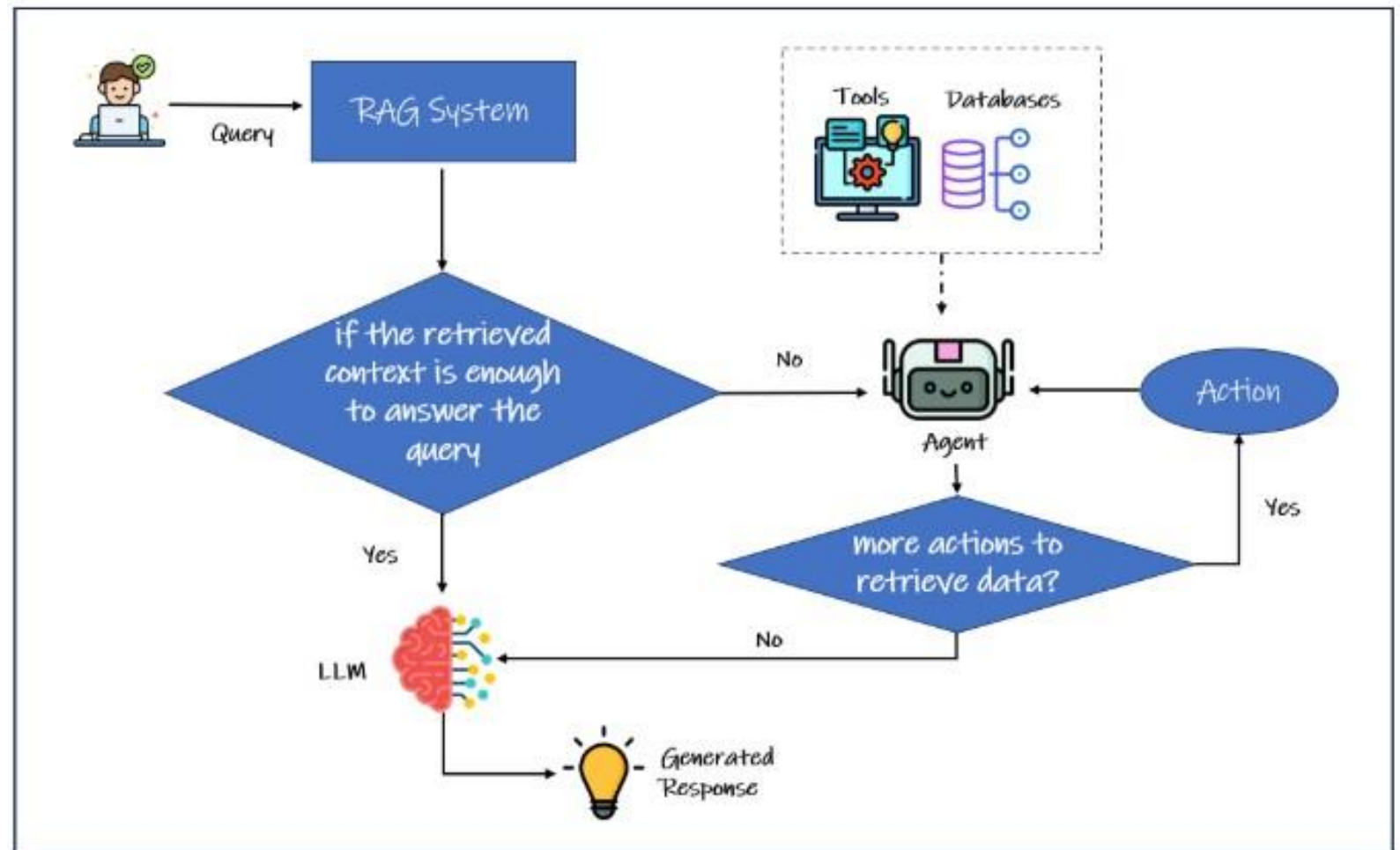


An agent is a system that decides what action to take next.

Agentic / tool-augmented RAG

Agentic or tool-augmented RAG extends basic RAG by allowing the model to do more than just retrieve documents.

- combines retrieval with tool use and decision-making
- does not always follow one fixed path
- can search, retrieve, calculate, route, and verify before answering



Agentic / tool-augmented RAG

Question:

“What was Q2 sales growth, and how does it compare to forecast?”

Possible workflow:

1. **retrieve** forecast document
 2. **query** sales database
 3. **calculate** difference
 4. generate answer
 5. **verify output format**
- 

Why agentic systems are attractive

- more flexible
- can solve more complex tasks
- can combine multiple sources
- can adapt to uncertain situations

But agents are risky

Cost of flexibility

- more latency
- more cost
- more moving parts
- harder debugging
- harder evaluation
- risk of loops or bad tool choices

Evaluate each step, not only the answer

A bad answer could come from:

- rewrite failure
- router failure
- retrieval failure
- tool failure
- reasoning failure
- verification failure

So evaluation should be component-level as well as end-to-end.

Next Week

- Responsible AI

Bias, fairness, Toxicity, Privacy

disinformation, hallucinations and uncertainty, safety alignment
strategies dataset governance

Build a small advanced RAG assistant with routing, tool use, and verification.

Additional Material:

- [*Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection.*](#)
- [Corrective Retrieval Augmented Generation \(CRAG\)](#)
- [Chain-of-Retrieval Augmented Generation \(CoRAG\)](#)
- [**Build a custom RAG agent with LangGraph**](#)

Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.



r.mu2@exeter.ac.uk

Questions?